Adaptive Homophily Clustering: Structure Homophily Graph Learning With Adaptive Filter for Hyperspectral Image

Yao Ding, Zhili Zhang, Weijie Kang, Aitao Yang, Junyang Zhao, Jie Feng, *Senior Member, IEEE*, Danfeng Hong, *Senior Member, IEEE*, and Qinghe Zheng

***Abstract*— Hyperspectral image (HSI) clustering is a fundamental yet challenging task that typically operates without training labels. Recent advancements in deep graph clustering methods have shown promise for HSI due to their ability to effectively encode spatial structural information. However, limitations such as inadequate utilization of structural information, poor feature representation, and weak graph update capabilities hinder their performance. In this article, we propose an adaptive homophily structure graph clustering (AHSGC) method for HSI. Our approach begins with the generation of homogeneous regions to process HSI and construct the initial graph. Next, we design an adaptive filter graph encoder that captures both high and low-frequency features for subsequent processing. We then develop a graph embedding clustering self-training decoder using KL Divergence to generate pseudo-labels for network training. To enhance graph learning, we introduce homophily-enhanced structure learning, which updates the graph based on the clustering task. This involves estimating node connections through orient correlation estimation and dynamically adjusting graph edges via graph edge sparsification. Finally, we implement joint network optimization to facilitate self-training and graph updates, with K-means used to express latent features. The clustering accuracy on three datasets is 83.60%, 63.65%, and 86.03%, the FLOPs are 3.57G, 30.62G, and 2.95G. The source code will be available at https://github.com/DY-HYX.**

***Index Terms*— Adaptive filter graph encoder, homophily-enhanced structure learning, hyperspectral image (HSI) clustering, joint network optimization.**

Received 13 December 2024; revised 31 January 2025 and 15 March 2025; accepted 26 March 2025. Date of current version 17 April 2025. This work was supported in part by the National Key Basic Research Strengthen Foundation of China under Grant 2021-JCJQ-JJ-0871 and in part by the National Natural Science Foundation of China under Grant 72401292. *(Corresponding authors: Zhili Zhang; Weijie Kang.)*

Yao Ding, Zhili Zhang, Weijie Kang, Aitao Yang, and Junyang Zhao are with the Key Laboratory of Intelligent Control, Xi'an-Research Institute of High Technology, Xi'an 710025, China (e-mail: dingyao.88@outlook.com; 157918018@qq.com; milankang@foxmail.com; 824360083@qq.com; zhaojy802@sina.com).

Jie Feng is with the Key Laboratory of Intelligent Perception and Image Understanding, Ministry of Education, Xidian University, Xi'an 710071, China (e-mail: jiefeng0109@163.com).

Danfeng Hong is with the Aerospace Information Research Institute, Chinese Academy of Sciences, Beijing 100094, China, and also with the School of Electronic, Electrical and Communication Engineering, University of Chinese Academy of Sciences, Beijing 100049, China (e-mail: hongdf@aircas.ac.cn).

Qinghe Zheng is with the School of Intelligent Engineering, Shandong Management University, Jinan 250357, China.

Digital Object Identifier 10.1109/TGRS.2025.3556276

NOMENCLATURE

$\mathcal{G} = (\mathcal{V}, \mathcal{E}, \boldsymbol{A})$	Graph $\mathcal{G}$ with edge $\mathcal{E}$ and node V.
N	Graph node number.
$\boldsymbol{X} \in \mathbb{R}^{N \times D}$	Node feature matrix.
$\boldsymbol{X}_i$	ith node feature.
$\boldsymbol{A} \in \mathbb{R}^{N \times N}$	Adjacency matrix.
S_i	ith superpixel.
n_i	Number of pixels contained in S_i.
$\boldsymbol{Z} \in \mathbb{R}^{hw \times N}$	Correlation matrix.
h, w, and b	Height, width, and number of bands of the HSI.
$\boldsymbol{S} \in \mathbb{R}^{N \times N}$	Node similarity matrix.
$\boldsymbol{H}$	Graph Laplacian filter.
$\boldsymbol{F}_A$	Adaptive graph filter.
$\boldsymbol{A}_{\text{rw}}$	Randomly wandering normalized affine matrix.
$\boldsymbol{Z}_i$ and $\boldsymbol{Z}$	ith layer and graph encoder output.
$\boldsymbol{Z} \in \mathbb{R}^{N \times D}$	Clustering-oriented node features.
ξ	Intracluster edge recovery ratio.
η	Intercluster edge removal ratio.
γ	High-confidence node extraction ratio.
$\mathcal{L}_c$, $\mathcal{L}_g$	Self-training loss and graph reconstruction loss.

I. INTRODUCTION

WITH the unique advantage of acquiring continuous spectral information for target objects, hyperspectral image (HSI) has been widely applied in numerous fields [1], [2], e.g., Earth observation [3], environmental monitoring [4], military detection [5], and resource ex spectral–spatial Transformer for spectral–spatial Transformer for ploration [6]. HSI contains rich potential for feature identification and classification, therefore, HSI interpretation has important research significance. However, the high dimensionality, strong redundancy, and high computational complexity contained by HSIs also present significant challenges for their analysis, processing, and interpretation [7], [8]. Traditionally, we often use numerous labeled samples to train interpretation methods and achieve HSI interpretation, which is costly, labor-intensive, and time-consuming [9], [10]. Therefore, the research on unsupervised HSI classification methods for land-cover classification analysis is of great significance. By grouping similar pixels

1558-0644 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:56:29 UTC from IEEE Xplore. Restrictions apply.

in HSI into distinct clusters and facilitating the extraction of meaningful information without pre-labeled training samples, Clustering analysis, an unsupervised learning technique, holds promise in overcoming these challenges [11], [12].

Prophase, some clustering methods initially developed for natural images have been adopted for HSI, including K-means clustering [13], possibilistic C-means (PCMs) [14], spectral clustering [15], fuzzy C-means (FCMs) [16], and density peak clustering [17]. These methods have achieved some success in HSI clustering. At the same time, they only utilize the prototypical features with numerous redundant information, facing challenges in effective dimension reduction, noise handling, and algorithm optimization [18]. Consequently, these limitations result in unideal clustering accuracy, and the widespread applicability has been restricted. After that, to address the redundant information interference, some subspace clustering (SC) algorithms [9] were designed by mapping the high-dimensional image information into a low-dimensional information space, such as sparse SC (SSC) [19], nonnegative matrix factorization (NMF) [20] and low-rank representation (LRR) [21]. However, these methods only focus on extracting spectral features, and the intrinsic relationship of spatially identifiable features is ignored [22]. To flexibly utilize spatial–spectral information, Zhang et al. [23] utilized a fuzzy similarity measure to identify the boundaries of different categories. Zhai et al. [24] further leveraged spatial information more effectively by adding an l_2 norm regularization based on SSC. Inspired by graph learning to extract potential relationships between nodes, some graph SC algorithms were developed. For example, Wang et al. [25] proposed a scalable graph-based clustering (SGCNR), in which the computational complexity is reduced by nonnegative relaxation. However, the aforementioned traditional clustering methods are shallow classifiers based on original features, and the handcrafted spectral–spatial features are often relied on, which are often unable to extract representative features [26]. These shortcomings make it difficult to explore deep semantic information and adapt to the spectral variations of HSI.

Inspired by the development of deep clustering learning, various deep clustering methods have been applied to overcome the limitations of traditional clustering methods, in which the hyper-parameters are updated through self-supervised training. The discriminative spatial–spectral features are learned for clustering. We often can divide most of the deep clustering methods into three modules, i.e., deep feature extractor, representative feature expressor, and self-training optimizer. For example, Zhang et al. [27] built spatial–spectral similarity graphs based on superpixels, and the spatial–spectral features were simultaneously extracted and decoded by a dual graph autoencoder. Cai et al. [18] proposed a contrastive SC method named NCSC, in which a pooling autoencoder was developed to learn the superpixel-level latent subspace and representation. These methods have made significant attempts to apply deep clustering methods to HSI clustering. At the same time, they are often needed more effectively capture intricate spatial structural information, resulting in subpar clustering performances.

Recently, the development of graph clustering methods has made it possible for deep clustering methods to effectively encode spatial structural information, which has transformed deep clustering methods from only processing Euclidean data to non-Euclidean geometric graph structure data. Numerous structural graph clustering networks have been developed, and they have demonstrated remarkable effectiveness in learning structural information through graph embeddings. For example, in [28], a low-pass graph filter is designed to learn smoother structural features, and the clustering accuracy is improved. Luo et al. [29] designed a dual transformer-based autoencoder to extract the long-dependency graph features, with which the graph structure features have been precisely presented. To explore target-oriented graph clustering methods, in [30] the graph attention network (GAT) was utilized to aggregate the neighbor node correlation information, and a better clustering performance was guided by developing self-training modules. Meanwhile, to extract higher-order and global structural correlations, some high-order graphs, e.g., pixel-superpixel graph [31], multiview graph, and hypergraph [32], have emerged to capture the structure and spectral–spatial information. Although we have made significant progress in graph-based clustering methods for HSI, the filters in existing graph networks are invariant and need more ability to process signal features for different nodes, resulting in poor clustering accuracy for large-scale HSI [33]. Moreover, mainstream graph networks typically do not allow for the updating of the original graph based on varying clustering tasks; instead, the graph remains fixed and unchanged throughout network training. This limitation makes it difficult to rectify erroneous edge connections in the original graph, significantly hindering the improvement of HIS clustering accuracy, especially in large graphs.

To address these challenges, we propose a novel method called adaptive homophily structure graph clustering (AHSGC) for HSIs. Our approach begins with a superpixel segmentation method to generate the graph structure. Next, we design a graph convolution encoder equipped with an adaptive filter that captures both high-frequency and low-frequency features of the graph in a dynamic manner. Additionally, we develop a graph embedding clustering self-training decoder to derive suitable graph representations. We also introduce a structure homophily-enhanced learning mechanism, which employs orient correlation estimation to assess pairwise correlations between nodes through a hierarchical correlation estimation framework. This is complemented by mechanisms for intracluster edge recovery and intercluster edge removal, allowing for dynamic adjustments to the graph structure based on specific clustering tasks. Subsequently, we implement joint network optimization, utilizing a self-training loss and a graph reconstruction loss to facilitate graph self-training and updates. Finally, we employ K-means clustering to represent the latent features. The effectiveness of the proposed sub-network lies in its ability to adaptively refine the graph structure in response to the clustering tasks, thereby enhancing the accuracy and robustness of HSI clustering. By dynamically adjusting the graph based on learned correlations and features, our method

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:56:29 UTC from IEEE Xplore. Restrictions apply.

can better capture the underlying relationships within the data, leading to improved clustering outcomes.

The innovative contributions of our AHSGC are as follows.

1) We propose a graph convolution encoder with an adaptive filter to extract both high-frequency and low-frequency components on the graph adaptively.
2) We develop homophily-enhanced structure learning to dynamically adjust the graph structure according to the different clustering tasks with intercluster edge removal and intracluster edge recovery
3) We adopt an auxiliary distribution to conduct the self-training for AHSGC, and then the k-means is utilized to implement clustering.

II. Notations and Definition

A. Graph and Graph Filter

1) Graph Definition: Let $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \boldsymbol{A})$ be an undirected graph with N nodes, C classes, and edge set $\mathcal{E}$, where $\mathcal{V} = \{v_1, v_2, \ldots, v_N\}$, $\boldsymbol{X} \in \mathbb{R}^{N\times D}$ and $\boldsymbol{A} \in \mathbb{R}^{N\times N}$ are the feature matrix and adjacency matrix of the constructed graph, respectively.

We summery the notions in Nomenclature.

2) Graph Filter Definition: For a discrete signal $\hat{f}_{\text{in}}(\xi)$, after the Fourier convolution operation [34], the output time-domain signal $f_{\text{out}}(t)$ can be expressed as

$$\begin{aligned} f_{\text{out}}(t) &= \int_{-\infty}^{+\infty} \hat{f}_{\text{in}}(\xi)\hat{h}(\xi)e^{2\pi i\xi t}d\xi \\ &= \int_{-\infty}^{+\infty} f_{\text{in}}(\tau)h(t-\tau)d\tau \\ &= (f_{\text{in}} * h)(t) \end{aligned} \tag{1}$$

where the unit of ξ is Hz, and $\hat{h}(\xi)$ is a frequency response function.

According to (1), the graph convolution operation on discrete graph node signal $\hat{h}(\lambda_k)$ can be defined as

$$f_{\text{out}}(i) = \sum_{k=1}^{n} \hat{f}_{\text{in}}(\lambda_k)\hat{h}(\lambda_k)\boldsymbol{v}_k(i) \tag{2}$$

where $\hat{f}_{\text{in}}(\lambda_k)$ represents the strength of the graph signal at frequency λ_k, $\boldsymbol{v}_k(\cdot)$ is the Fourier basis of the graph signal, i represents the ith element of $\boldsymbol{v}_k(\cdot)$. In (2), the integral operation in (1) is replaced with the sum operation.

Let $\hat{f}_{\text{in}} = U^{\text{T}} f_{\text{in}}$, we can simplify (2) to matrix form as

$$\begin{aligned} f_{\text{out}} &= \sum_{k=1}^{n} h(\lambda_k)\hat{f}_{\text{in}}(\lambda_k)\boldsymbol{v}_k \\ &= \begin{bmatrix} | & | & \cdots & | \\ \boldsymbol{v}_1 & \boldsymbol{v}_2 & \cdots & \boldsymbol{v}_n \\ | & | & \cdots & | \end{bmatrix} \begin{bmatrix} h(\lambda_1)\hat{f}_{\text{in}}(\lambda_1) \\ h(\lambda_2)\hat{f}_{\text{in}}(\lambda_2) \\ \cdots \\ h(\lambda_n)\hat{f}_{\text{in}}(\lambda_n) \end{bmatrix} \\ &= \begin{bmatrix} | & | & \cdots & | \\ \boldsymbol{v}_1 & \boldsymbol{v}_2 & \cdots & \boldsymbol{v}_n \\ | & | & \cdots & | \end{bmatrix} \begin{bmatrix} h(\lambda_1) & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & h(\lambda_n) \end{bmatrix} \\ &\quad \times \begin{bmatrix} - & \boldsymbol{v}_1^{\text{T}} & - \\ - & \boldsymbol{v}_2^{\text{T}} & - \\ - & \cdots & - \\ - & \boldsymbol{v}_n^{\text{T}} & - \end{bmatrix} f_{\text{in}} \\ &= \boldsymbol{U}\begin{bmatrix} h(\lambda_1) & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & h(\lambda_n) \end{bmatrix} \boldsymbol{U}^{\text{T}} f_{\text{in}} = \boldsymbol{U}\boldsymbol{\Lambda}_h\boldsymbol{U}^{\text{T}} f_{\text{in}} \\ &= \mathbf{H} f_{\text{in}}. \end{aligned} \tag{3}$$

Accordingly, the graph filter can be defined as

$$\boldsymbol{H} = \boldsymbol{U}\boldsymbol{\Lambda}_h\boldsymbol{U}^{\text{T}} \in \mathbb{R}^{n\times n}, \quad \boldsymbol{H} : \mathbb{R}^n \to \mathbb{R}^n \tag{4}$$

where $\boldsymbol{\Lambda}_h$ is the frequency response matrix of the graph filter.

B. Homophily-Enhanced Graph Learning

Given a graph $\mathcal{G} = \{\boldsymbol{X}, \boldsymbol{A}\}$ with C classes, we assign the node $\boldsymbol{v}_i$ as y_i. The edge homogeneity concept is the ratio of edges in the same class to the total number of edges in the graph, it can be defined as

$$h(\mathcal{G}, \{y_i; v_i \in \mathcal{V}\}) = \frac{1}{|\mathcal{E}|} \sum_{(v_i, v_j)\in\mathcal{E}} \mathbb{I}(y_i = y_j) \tag{5}$$

where $\mathbb{I}(\cdot)$ is an indicator function

The purpose of graph clustering is to classify node set $\mathcal{V}$ into class C, that is, to assign node $\boldsymbol{v}_i$ into specific cluster with its specific features, which can be expressed as

$$\begin{cases} \min \sum_{i=1}^{m}\sum_{j=1}^{m} d(i,j) = \min \sum_{i=1}^{m}\sum_{j=1}^{m} \left\| f(\boldsymbol{x}_i) - f(\boldsymbol{x}_j) \right\|_2^2 \\ \max \sum_{\alpha=1}^{c}\sum_{\beta=1}^{c} d(\alpha,\beta) = \max \sum_{\alpha=1}^{c}\sum_{\beta=1}^{c} \left\| y(\boldsymbol{c}_\alpha) - y(\boldsymbol{c}_\beta) \right\|_2^2 \end{cases} \tag{6}$$

where $f(\cdot)$ and $y(\cdot)$ are functions to map the node's optimal spatial–spectral information and calculate the feature of the cluster, respectively. m is the node number within the specific cluster.

III. Proposed Methodology

In this section, the overall framework of AHSGC is elaborated first. Afterward, HSI Preprocessing and Initial Graph Construction are introduced. Finally, the structure details of AHSGC, i.e., adaptive filter graph encoder, graph embedding clustering self-training decoder, homophily-enhanced structure learning, and joint network optimization are fully introduced.

A. Overall AHSGC Framework

The proposed AHSGC mainly consists of five modules, i.e., adaptive filter graph encoder, graph embedding clustering self-training decoder, homophily-enhanced structure learning, joint network optimization, and HSI preprocessing and initial graph construction module, the overall framework of AHSGC is shown in Fig. 1. The main functions of the five modules are as follows.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:56:29 UTC from IEEE Xplore. Restrictions apply.

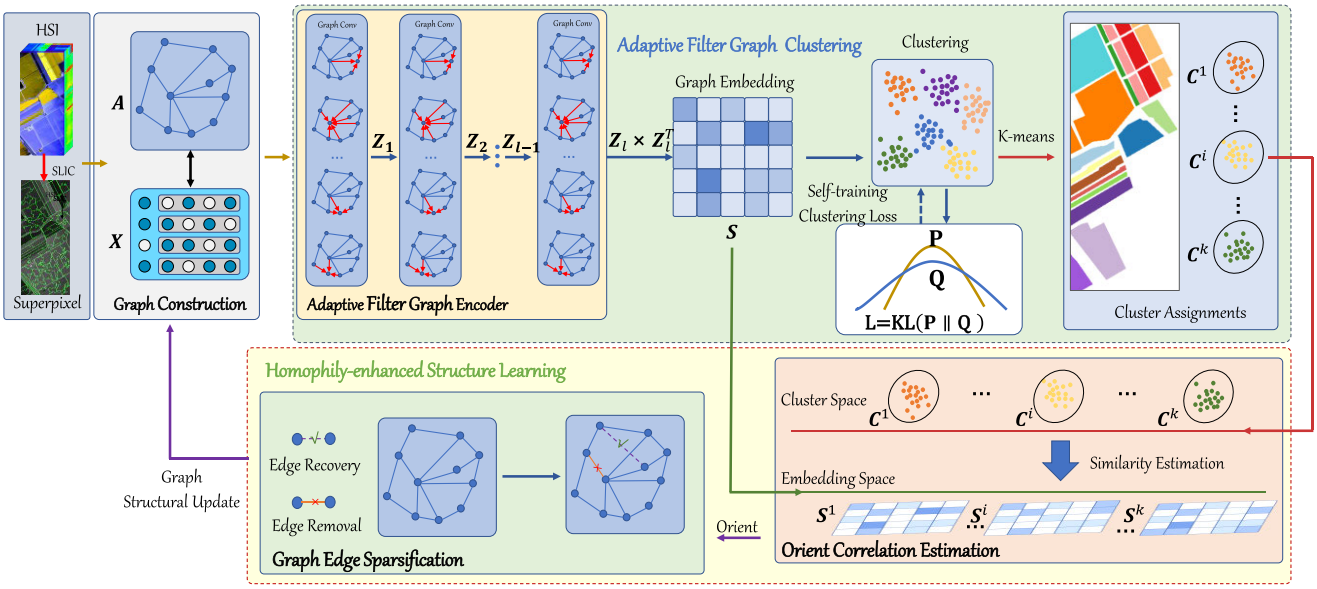


Fig. 1. Overview of AHSGC for HSI. AHSGC mainly consists of five parts, i.e., HSI Preprocessing and initial graph construction, adaptive filter graph encoder, graph embedding clustering self-training decoder, homophily-enhanced structure learning, and joint network optimization module. The HSI Preprocessing and initial graph construction are proposed to transform the pixels in HSI to locality-preserving regions and construct the original graph. An adaptive filter graph encoder is designed to extract and filter the low and high frequency features on the graph. The graph embedding clustering self-training decoder is used to conduct the self-training and decoder the extracted features. The homophily-enhanced structure learning is developed to update the graph according to the clustering task. The joint network optimization module is proposed to enhance the structural consistency and graph edge sparsification of the graph and train the proposed network.

1) *HSI Preprocessing and Initial Graph Construction:* A superpixel segmentation method is established to transform the HSIs from pixels to regions while preserving the local spatial–spectral structure information and reducing the node number.
2) *Adaptive Filter Graph Encoder:* The superpixel-level graph is the input of AHSGC. To extract and filter the low and high-frequency features on the graph for clustering, an adaptive graph filter is designed by conducting neighbor information aggregation with an adaptive graph filter.
3) *Graph Embedding Clustering Self-Training Decoder:* An auxiliary distribution is adopted to conduct the self-training for AHSGC, and then the k-means is utilized to implement clustering.
4) *Homophily-Enhanced Structure Learning:* To obtain sparser edges, we developed intercluster edge removal and intracluster edge recovery, with which the graph structure can be dynamically adjusted according to the different clustering tasks.
5) *Joint Network Optimization:* A novel joint objective loss is proposed to enhance the structural consistency and graph edge sparsification of the graph and train the proposed network. Finally, the K-means is adopted to express the latent features.

In our AHSGC, five clustering modules are joined in an end-to-end network for HSI clustering, and each portion interacts with the other.

B. HSI Preprocessing and Initial Graph Construction

In our paper, the simple linear iterative cluster (SLIC) [35] method is first applied to divide the entire HSI into many spatially connected superpixels. Furthermore, the superpixel-level node feature $\boldsymbol{X}$ is calculated as the average spectral features of the pixels contained in that superpixel. The $\boldsymbol{A} \in \mathbb{R}^{N\times N}$ is formulated as

$$\boldsymbol{A}_{ij} = \begin{cases} e^{-\rho\|\boldsymbol{X}_i-\boldsymbol{X}_j\|^2}, & \text{if } \boldsymbol{X}_i \in N_t(\boldsymbol{X}_j) \text{ or } \boldsymbol{X}_j \in N_t(\boldsymbol{X}_i) \\ 0, & \text{otherwise} \end{cases} \tag{7}$$

where $\boldsymbol{X}_i$ denotes ith superpixel-level node feature, $\rho = 0.2$, $N_t(\boldsymbol{X}_i)$ presents the node $v_i's$ t-hop neighbors.

To explore the connection between pixel and superpixel, image backprojection operations are developed to transform data features from pixel to superpixel, it can be represented as

$$\boldsymbol{Q}_{i,j} = \begin{cases} 1, & \text{if } x_i \in S_i \\ 0, & \text{otherwise} \end{cases} \tag{8}$$

where $\boldsymbol{Q} \in \mathbb{R}^{hw\times N}$ is the correlation matrix to record all pixel contained in HSI locating in which superpixel.

Graph projection can encode the original pixel level HSI into superpixel-level graph node features through matrix multiplication, which is formulated as

$$\boldsymbol{V} = \text{Projection}(\boldsymbol{X}; \boldsymbol{Q}) = \hat{\boldsymbol{Q}}^{\text{T}}\text{Flatten}(\boldsymbol{X}) \tag{9}$$

where $\hat{\boldsymbol{Q}}$ is $\boldsymbol{Q}$ normalized by column, i.e., $\hat{\boldsymbol{Q}}_{i,j} = \boldsymbol{Q}_{i,j}/\sum_m \boldsymbol{Q}_{m,j}$. Flatten$(\cdot)$ represents flattening HSI according to spatial dimensions.

C. Adaptive Filter Graph Encoder

Typically, the low-pass filters can obtain low-frequency information on the graph, filtering out high-frequency information, and the smoother graph node signal features are

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:56:29 UTC from IEEE Xplore. Restrictions apply.

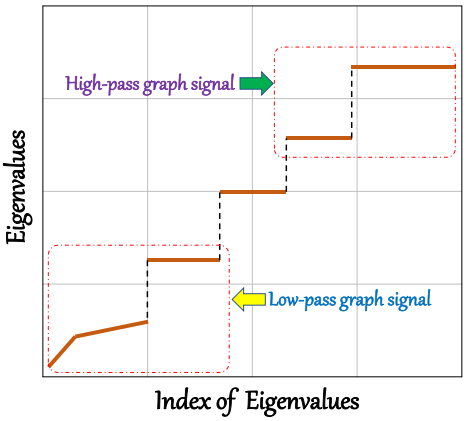


Fig. 2. Illustration of the role of low-pass and high-pass filters.

obtained. While the high-pass filters have the opposite function (shown in Fig. 2). Importantly, the graph node signals contain low-pass signals and high pass signals, both of which have significant impacts on downstream clustering tasks [36]. How to comprehensively utilize the two types of feature signals is of great significance for improving clustering accuracy.

In general, graph filters are associated with Laplacian matrices, affinity matrices, and their variants, respectively. The low-pass filters $\boldsymbol{H}_l$ and high-pass filters $\boldsymbol{H}_h$ can be expressed as

$$\boldsymbol{H}_l = \boldsymbol{A}_{\mathrm{rw}}\boldsymbol{X}, \quad \boldsymbol{H}_h = \boldsymbol{L}_{\mathrm{rw}}\boldsymbol{X} \tag{10}$$

where $\boldsymbol{A}_{\mathrm{rw}}$ is the randomly wandering normalized affine matrix, expressed as $\boldsymbol{A}_{\mathrm{rw}} = \boldsymbol{D}^{-1}\boldsymbol{A}$, $\boldsymbol{L}_{\mathrm{rw}}$ is the corresponding Laplacian matrix, namely $\boldsymbol{L}_{\mathrm{rw}} = \boldsymbol{I} - \boldsymbol{A}_{\mathrm{rw}}$.

According to the introduction in Section II-A, given a convolution kernel f and a graph signal x, the convolution operation can be expressed as follows:

$$f * x = \boldsymbol{U}\left(\left(\boldsymbol{U}^{\mathrm{T}} f\right) \odot \left(\boldsymbol{U}^{\mathrm{T}} x\right)\right) = \boldsymbol{U} g^{\theta} \boldsymbol{U}^{\mathrm{T}} x \tag{11}$$

where $g^{\theta} = \boldsymbol{U}^{\mathrm{T}} f$ denotes the representation of f in the spectral domain, $\odot$ is the matrix dot multiplication.

If $\boldsymbol{A}_{\mathrm{rw}}$ and $\boldsymbol{L}_{\mathrm{rw}}$ are treated as convolution kernels as (11), then we can filter the graph signal x as

$$\boldsymbol{A}_{\mathrm{rw}} * x = \boldsymbol{U}\left(g_l^{\theta}\right)\boldsymbol{U}^{\mathrm{T}} x, \quad \boldsymbol{L}_{\mathrm{rw}} * x = \boldsymbol{U}\left(g_h^{\theta}\right)\boldsymbol{U}^{\mathrm{T}} x \tag{12}$$

where g_l^{θ} and g_h^{θ} are the representation of $\boldsymbol{A}_{\mathrm{rw}}$ and $\boldsymbol{L}_{\mathrm{rw}}$ in the spectral domain. If $g_l^{\theta} = \boldsymbol{I} - \boldsymbol{\Lambda}$ and $g_h^{\theta} = \boldsymbol{\Lambda}$, (12) is rewrite as

$$\begin{aligned} \boldsymbol{U}\left(g_l^{\theta}\right)\boldsymbol{U}^{\mathrm{T}} x &= \sum_i \left(1 - \lambda_i\right) u_i u_i^{\mathrm{T}} x \\ \boldsymbol{U}\left(g_h^{\theta}\right)\boldsymbol{U}^{\mathrm{T}} x &= \sum_i \lambda_i u_i u_i^{\mathrm{T}} x. \end{aligned} \tag{13}$$

In (13), when the eigenvector $\lambda > 1$, the differences among the elements x_i are amplified in the projection $\sum_i \lambda_i u_i u_i^{\mathrm{T}} x$, resulting in the extraction of non-smooth signals. Conversely, if $\lambda < 1$, the differences among x_i are diminished in the projection $\sum_i \lambda_i u_i u_i^{\mathrm{T}} x$, leading to the acquisition of smooth signals. Our analysis indicates that low-pass filters are effective in capturing similarity information, while high-pass filters serve the opposite function. Consequently, both filter types are crucial in graph networks. To fully capture node information, we propose an adaptive graph filter $\boldsymbol{F}_A$, defined as

$$\boldsymbol{F}_{\mathrm{A}} = \mu \cdot (\boldsymbol{S}_{\mathrm{rw}})^k \boldsymbol{X} + (1 - \mu) \cdot (\boldsymbol{I} - \boldsymbol{S}_{\mathrm{rw}})^k \boldsymbol{X} \tag{14}$$

where $\boldsymbol{X}$ denotes the input spectral features, and $\mu > 0$ is a learnable parameter that balances high-pass and low-pass information. $\boldsymbol{S}_{\mathrm{rw}}$ refers to the normalized Laplacian matrices associated with $\boldsymbol{A}_{\mathrm{rw}}$.

To obtain both high-pass and low-pass representations of the constructed graph, we stack t layers of graph Laplacian filters as follows:

$$\boldsymbol{X}_s = \left(\prod_{i=1}^{t} \boldsymbol{F}_A\right)\boldsymbol{X} = \boldsymbol{F}_A^t \boldsymbol{X} \tag{15}$$

where $\boldsymbol{X}$ represents the input graph node spectral features, $\boldsymbol{F}_A^t$ denotes the stacked t layers of adaptive graph filters, and $\boldsymbol{X}_s$ signifies the graph node presentations after t layers convolution.

D. Graph Embedding Clustering Self-Training Decoder

1) Graph Embedding: The adaptive filter graph encoder module extracts high-level structural-semantic information, while the high-pass and low-pass presentations are learned. To achieve an efficient fusion representation of structural information and spectral features, a linear transformation mechanism is developed to encode the graph structure as follows:

$$\boldsymbol{Z} = \left(\prod_{i=1}^{t} \boldsymbol{F}_A\right)\boldsymbol{Z}_0 \boldsymbol{W} = \boldsymbol{F}_A^t \boldsymbol{Z}_0 \boldsymbol{W} \tag{16}$$

where $\boldsymbol{Z}_i$ and $\boldsymbol{Z}$ are the ith layer and graph encoder output, respectively, $\boldsymbol{Z}_0 = \boldsymbol{X} \cdot \boldsymbol{W}$ denotes the linear transformation weight matrix.

2) Self-Training Clustering: As analyzed above, we can learn a high-pass and low-pass graph presentations with an adaptive filter. However, the high-level structural-semantic information cannot be used for clustering, and unlabeled is also a barrier to graph clustering. To address the problems, a self-training clustering mechanism is designed by using an auxiliary distribution. Concretely, we adopt a Student's t-distribution to measure the similarity between the kth cluster center $\boldsymbol{\mu}_k$ and the node embedding $\boldsymbol{z}_i$ as follows:

$$q_{ik} = \frac{\left(1 + \|\boldsymbol{z}_i - \boldsymbol{\mu}_k\|^2\right)^{-1}}{\sum_k^K \left(1 + \|\boldsymbol{z}_i - \boldsymbol{\mu}_k\|^2\right)^{-1}} \tag{17}$$

where q_{ik} is clustering assignment distribution, which can be considered as the soft assignment of node v_i to $\boldsymbol{\mu}_k$. Then we can optimize the clustering distribution by minimizing the KL divergence, namely

$$\mathcal{L}_c = \mathrm{KL}(\boldsymbol{P} \| \boldsymbol{Q}) = \sum_i \sum_k p_{ik} \log \frac{p_{ik}}{q_{ik}} \tag{18}$$

$$p_{ik} = \frac{q_{ik}^2 \Big/ \sum_j q_{jk}}{\sum_k^K \left(q_{ik}^2 \Big/ \sum_j q_{jk}\right)} \tag{19}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:56:29 UTC from IEEE Xplore. Restrictions apply.

where p_{ik} is the auxiliary distribution, as described in (16), the $\boldsymbol{P}$ raises $\boldsymbol{Q}$ to the second power, with which the $\boldsymbol{Q}$ would be a sharp distribution, and it can avoided the clustering collapsing into a single cluster [37].

E. Homophily-Enhanced Structure Learning

1) Orient Correlation Estimation: Given an intermediate clustering result, generally, the inner product is used to measure the pairwise correlation between nodes. While, in practice, we have found that this is a suboptimal approach, because the clustering results contain considerable noise during the network training [38]. To address this problem, a hierarchical correlation estimation mechanism is designed. Precisely, the node similarity is estimated incorporating guidance from both the latent space in a hierarchical manner and clustering. First, the hard pseudo-label is calculated

$$c_i = \arg\max_k q_{ik}. \tag{20}$$

According to (20), all nodes' pseudo-labels are obtained, then, the nodes' top γ percentage is selected in each cluster k with the highest assignment probability. Thus, a node subset is obtained, namely

$$\Gamma^k = \left\{ v_i | c_i = k, \text{Rank}|_{\{v_i|c_i=k\}}(q_{ik}) \leq \gamma * |\{v_i|c_i = k\}| \right\} \tag{21}$$

where Γ^k is the top-confident node subset, $\text{Rank}_{\{v_i|c_i=k\}}(q_{ik})$ denotes the cluster assignment ranking probability of node v_i in cluster k. With (21), we can reduce the noise effects on node clustering. However, if the edge weight is still determined by cluster space, confident but incorrect segments will still undermine the similarity matrix accuracy. To overcome the limitations, in this article, we adopt the latent space to estimate the node similarity rather than the clustering space.

In (16), the node embedding matrix $\boldsymbol{Z}$ is calculated, we can estimate the similarity between all nodes by dot product

$$\boldsymbol{S} = \boldsymbol{Z}\boldsymbol{Z}^{\mathrm{T}} \tag{22}$$

where $\boldsymbol{S}$ is node similarity. Different from (22), in this article, we calculate $\boldsymbol{S}$ within Γ^k, and the corresponding node embedding node similarity $\boldsymbol{S}_{ij}^k$ is expressed as

$$\boldsymbol{S}_{ij}^k = \boldsymbol{Z}_i^k \boldsymbol{Z}_j^{k^{\mathrm{T}}} \in \mathbb{R}^{|\Gamma^k| \times |\Gamma^k|} \tag{23}$$

where $\boldsymbol{Z}_i^k$ is the node embedding matrix of ith class, $|\Gamma^k|$ denotes the number of nodes in Γ^k.

Equation (23) comprehensively utilizes the embedding similarity matrix and soft assignments to identify the incorrect or missing edges, with which the edge modification accuracy is significantly enhanced. Therefore, a more reliable and precise graph structure learning is achieved.

2) Graph Edge Sparsification: Most existing methods keep the graph unchanged during the network training, which means that the edges in the graph remain unchanged. This strategy has two drawbacks, e.g., incorrect edge connections in the graph cannot be corrected, and it brings unnecessary difficulties to clustering. Inspired by (6), we propose a graph edge sparsification, including intercluster edge removal and intracluster edge recovery, to make the nodes within the cluster more closely connected and the nodes between clusters sparser (shown in Fig. 3).

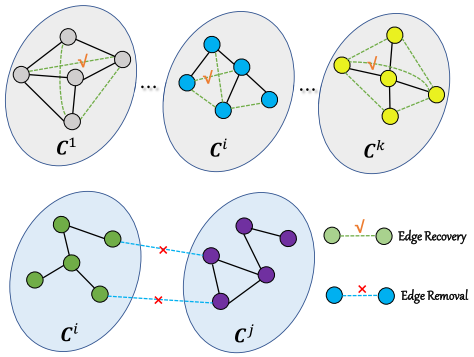


Fig. 3. Mechanism of intracluster edge recovery and Intercluster edge removal.

a) Intracluster edge recovery: To increase connections within the cluster, the edges are added to the top ξ percent of node pairs containing the largest similarity $\boldsymbol{S}_{ij}^k$ within cluster k, namely

$$\mathcal{E}_{\text{rc}}^k = \left\{ (v_i, v_j) \middle| \text{Rank}(S_{ij}^k) \leq \xi * |\mathcal{E}| * \frac{N_k}{N}, \text{ and } c_i = c_j = k \right\} \tag{24}$$

where N_k denotes the node number in Γ^k, $\mathcal{E}$ is the edge-set on the current graph learning, $\text{Rank}(S_{ij}^k)$ indicates the node pair (v_i, v_j) similarity is ranked in order within $\boldsymbol{S}^k$. We should note that existing edges are unconsidered in the ranking process. We can express the complete intracluster edge recovery set $\mathcal{E}_{\text{rc}}$ as

$$\mathcal{E}_{\text{rc}} = \bigcup_k^K \mathcal{E}_{\text{rc}}^k. \tag{25}$$

b) Intercluster edge removal: This strategy aims to remove unnecessary edge connections between clusters, weaken inter cluster connections, and facilitate clustering. The expression is as follows:

$$\mathcal{E}_{rm} = \left\{ (v_i, v_j) \middle| \text{Rank}(S_{ij}) \geq (1-\eta) * |\mathcal{E}| (v_i v_j) \in \mathcal{E}, c_i \neq c_j \right\} \tag{26}$$

where $\mathcal{E}$ is the edge-set on the current graph learning. η is the percentage of retained edges.

c) Homophily-enhanced structure learning: We have demonstrated the recovery and removal edge set in (25) and (26). The edge set during the graph learning can be expressed as $\bar{\mathcal{E}} = \mathcal{E} - \mathcal{E}_{rm} + \mathcal{E}_{\text{rc}}$. The adjacency matrix $\bar{\boldsymbol{A}}$ can be computed as

$$\bar{\boldsymbol{A}} = \boldsymbol{A} - \boldsymbol{A}_{\mathcal{E}_{rm}} + \boldsymbol{A}_{\mathcal{E}_{\text{rc}}} \tag{27}$$

where $\boldsymbol{A}_{\mathcal{E}_{rm}}$ and $\boldsymbol{A}_{\mathcal{E}_{\text{rc}}}$ are the adjacency matrices of $\mathcal{E}_{rm}$ and $\mathcal{E}_{\text{rc}}$. Then, we reconstruct the graph by updating the adjacency matrix, namely

$$\mathcal{L}_g = \|\boldsymbol{Z}\boldsymbol{Z}^{\mathrm{T}} - \bar{\boldsymbol{A}}\|_F^2 \tag{28}$$

where $\mathcal{L}_g$ is the graph reconstruction loss.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:56:29 UTC from IEEE Xplore. Restrictions apply.

This hierarchical mechanism addresses the limitations of static graphs by dynamically refining the graph structure, ensuring that nodes within clusters are more tightly connected while reducing unnecessary connections between clusters.

F. Joint Network Optimization

In the proposed AHSGC, the self-training clustering loss and the structure graph reconstruction loss are shown in (18) and (28), respectively. We use two loss functions to self-train the network and reconstruct the graph separately, then the overall objective $\mathcal{L}_O$ of AHSGC can be formulated as

$$\mathcal{L}_O = \mathcal{L}_c + \mathcal{L}_g. \tag{29}$$

In AHSGC, the p_{ik} is treated as "ground-truth," and the hyper-parameters are updated T iteration by minimizing the overall objective $\mathcal{L}_O$. The clustering result for node v_i is formulated as

$$Y_i = \arg\max_k q_{ik}. \tag{30}$$

Therefore, the adaptive filter graph encoder is used to capture the latent features of the graph, and the graph embeddings $\boldsymbol{Z}$ is derived. Subsequently, the K-means is adapted to perform clustering on $\boldsymbol{Z}$ based on the initialized cluster centers $\boldsymbol{\mu}_k$. The learning process is detailed in Algorithm 1.

Algorithm 1 AHSGC

Input: The input graph; Initial centroids (Cluster) number k; Iterations number T; Number of the adaptive graph filter layer t.
Output: Clustering results $\boldsymbol{Q}$.

Initialization: t; $\boldsymbol{Z}^0 = \boldsymbol{X}$;

Generate the homogeneous region and construct the superpixel-based graph;
for $t = 1$ to T **do**
  Apply t-layers graph convolution with adaptive graph filters on $\boldsymbol{Z}^0$ to obtain the latent $\boldsymbol{Z}$ by Eq. (16);
  The self-training clustering loss $\mathcal{L}_c$ is formulated as by Eq. (18);
  The homophily-enhanced structure learning is implemented by Eq. (24) and Eq. (26);
  The adjacency matrix $\bar{\boldsymbol{A}}$ is calculated by Eq. (27);
  The graph reconstruction loss is calculated by Eq. (28)
  The weight matrices are updated by minimizing $\mathcal{L}_O$ with Adam optimizer.
end
The clustering results $\boldsymbol{Q}$ is obtained by applying K-means on $\boldsymbol{Z}$.

IV. Experiments

In this section, extensive experiments and analyses are introduced to assess the clustering performance of the proposed AHSGC. Specifically, the three well-known HSI datasets and experimental setups are introduced (see Sections IV-A and IV-B); subsequently, the quantitative and visual HSI clustering performances of AHSGC are compared with nine state-of-the-art algorithms (see Sections IV-C and IV-D); furthermore, the runtime and model complexity are compared with investigated methods (see Section IV-E); in addition, the influence of hyperparameters are analyzed (see Section IV-F); moreover, the clustering visualizations are demonstrated (see Section IV-G), finally, we implement some ablation experiments to evaluate the rationality of the proposed AHSGC design (see Section IV-H).

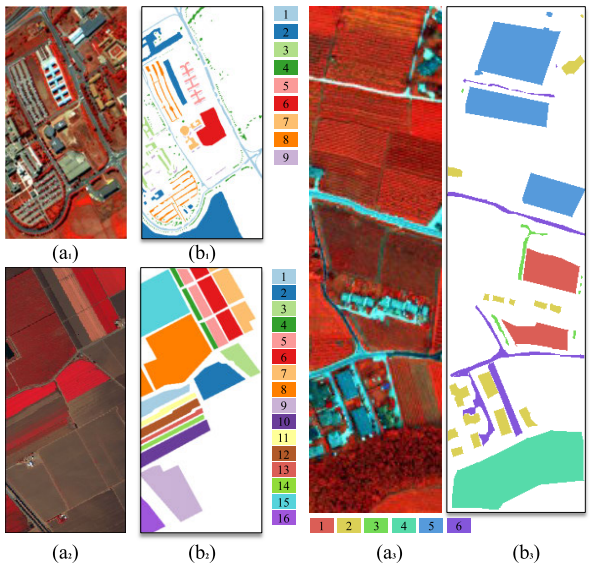


Fig. 4. Three dataset details. (a1)–(a3) False-color maps of SA, PU, and Trento datasets. (b1)–(b3) Ground-truth maps.

A. Data Description

In this article, we validate our method using the Salina (SA), Trento, and Pavia University (PU) datasets. The SA dataset contains a dimension of 512 × 217 and 204 bands, totaling 16 land cover classes. The Trento dataset has an image size of 600 × 166, with 63 spectral channels ranging from 0.40 to 0.98 μm, covering six land cover classes. The PU dataset consists of an image composed of 103 spectral bands with dimensions of 610 × 340, encompassing nine land cover classes. We show the specific land covers and corresponding quantities of the three datasets in Table I, and the corresponding ground-truth is shown in Fig. 4.

B. Experimental Setup

1) Experiment Settings: The overall classification accuracy (OA), kappa coefficient (κ), normalized mutual information (NMI), adjusted rand index (ARI), and Purity are adopted as quantitative evaluation metrics. Among them, OA is used to record the overall clustering accuracy; κ is an indicator adopted to evaluate the consistency degree; NMI denotes the similarity of clustering results; ARI represents the degree of agreement between clustering results and the ground truth, Purity $\in$ [0, 1] is a simple and transparent evaluation metric. In our AHSGC, seven main parameters, i.e., graph number N,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:56:29 UTC from IEEE Xplore. Restrictions apply.

TABLE I
DETAILS OF EACH LAND-COVER CLASS OF THREE DATASETS USED IN THIS ARTICLE

NO.	SA		PU		Trento	
	Name	Pixels	Name	Pixels	Name	Pixels
1	Weed 1	2009	Asphalt	6631	Apple trees	4034
2	Weed 2	3726	Meadows	18649	Buildings	2903
3	Fallow	1976	Gravel	2099	Ground	479
4	Fallow-plow	1394	Trees	3064	Woods	9123
5	Fallow-smooth	2678	Painted metal sheets	1345	Vineyard	10501
6	Stubble	3959	Bare Soil	5029	Roads	3174
7	Celery	3579	Bitumen	1330		
8	Grapes untrained	11271	Self-Blocking Bricks	3682		
9	Soil	6203	Shadows	947		
10	Corn	3278				
11	Lettuce-4wk	1068				
12	Lettuce-5wk	1927				
13	Lettuce-6wk	916				
14	Lettuce-7wk	1070				
15	Vineyard-untrained	7268				
16	Vineyard-trellis	1807				
Total		54129		42776		30214

TABLE II
PRESET OPTIMAL PARAMETERS IN AHSGC

Dataset	N	l	T	L	γ	ξ	η
Salinas	580	5	50	5e-4	0.3	0.5	0.05
PU	800	5	50	5e-4	0.5	0.5	0.05
Trento	550	5	50	5e-4	0.3	0.5	0.05

encoder layer number l, iterations number T, learning rate L, intracluster edge recovery ratio ξ, intercluster edge removal ratio η, and high-confidence node extraction ratio γ should be preset. Their optimal values are shown in Table II and will be analyzed in Section IV-F.

2) Compare Baseline: To comprehensively analyze the effectiveness of the proposed method, we compare AHSGC with nine selected baselines, e.g., K-means, FCMs, PCMs, NCSC [18], DFCN [39], SDCN [40], EGAE [41], AdaGAE [42], and DAEGC [30]. Among them, K-means, FCM, and PCM are traditional clustering methods, NCSC and DAEGC are SC and graph clustering methods, DFCN, SDCN, EGAE, AdaGAE, and DAEGC are deep clustering methods, and DAEGC, EGAE, and AdaGAE are graph autoencoder methods. The hyper-parameters and network structures of these methods follow the suggestions in the original papers.

3) Implementation Details: In this article, all investigated experiments are conducted on NVIDIA Titan RTX using the Pytorch framework. For a fair comparison, all methods are executed ten times for the sake of eliminating any bias caused by the random selection of training samples.

C. Quantitative and Qualitative Performances Comparison

In this section, we compare AHSGC with nine other clustering methods to demonstrate its clustering performance through quantitative and qualitative performance analyses. In addition, to be more intuitive, the best performances and the second-best performances are marked in bold and underlined.

1) Quantitative Analysis of the SA Dataset: The different clustering methods' performances on SA are shown in Table III. From Table III, we can note that our AHSGC achieves the best clustering performance with 83.60%, 81.62%, 77.42%, and 83.68% in terms of OA, Kappa, NMI, ARI, and Purity, respectively, which is improved by 4.47%, 5.60%, 0.73%, 5.59%, and 0.11% compared with the second-best results. Moreover, NCSC, EGAE, and AdaGAE achieve relatively good clustering results, which show that these methods have good adaptability to noise interference and class imbalance sample clustering. Moreover, due to the remarkable ability of graph-based methods to exploit the interrelationships between nodes, DAEGC, EGAE, and AdaGAE achieve relatively good clustering results. Due to lacking the ability to capture high-lever features of HSI, the traditional methods, including k-mean, FCM, and PCM) undoubtedly achieve poor results. Specifically, the OAs of the three methods are 67.99%, 56.73%, and 55.63%, respectively. Furthermore, compared with other graph-based methods, an adaptive filter can adaptively extract the high-frequency and low-frequency information for subsequent HSI clustering tasks, improving the performance of AHSGC.

2) Quantitative Analysis of the PU Dataset: Compared to the SA dataset, the land cover types in the PU dataset are more dispersed, posing significant challenges for clustering. From the clustering results in Table IV, it can also be concluded that the clustering results of all comparison methods have decreased. However, our AHSGC method still performs the best among all investigated clustering methods. Notably, compared with the best results in the comparative methods, the performances of AHSGC are improved by 3.55% in OA, 1.83% in κ, and 0.77% in ARI, respectively. In addition, we note that deep and graph-based clustering methods do not have significant advantages over traditional clustering methods, mainly because the land features in the PU dataset are scattered and closely related, making it difficult for clustering methods to distinguish. Therefore, it is more important to extract more significant and representative features. It is worth

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:56:29 UTC from IEEE Xplore. Restrictions apply.

TABLE III
QUANTITATIVE EXPERIMENTAL CLASSIFICATION RESULTS ON SA

No.	k-means	FCM	PCM	NCSC	DFCN	SDCN	EGAE	AdaGAE	DAEGC	AHSGC
1	0.9985	0.4179	0.6456	0.0000	**1.0000**	0.0000	**1.0000**	0.0000	**1.0000**	**1.0000**
2	0.5698	0.5524	0.8125	**1.0000**	0.9990	**1.0000**	**1.0000**	0.9997	0.9997	**1.0000**
3	0.9741	0.8501	0.0000	0.6822	**1.0000**	0.3426	**1.0000**	**1.0000**	0.0000	0.9413
4	**0.9865**	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
5	0.7856	0.5540	0.9628	**0.9988**	0.7308	0.9816	0.0000	0.9572	0.6502	0.9709
6	0.9952	0.9990	0.9906	0.9941	0.9986	0.9530	0.9886	0.9900	**1.0000**	0.9914
7	0.4810	0.0000	0.9943	**1.0000**	0.9561	0.0000	0.9983	0.9999	0.9718	0.9922
8	0.7309	0.7772	0.3875	0.8026	0.4320	0.7633	0.5131	0.5631	0.6683	**0.9403**
9	0.9750	0.6969	0.5829	0.9076	0.9063	0.8643	0.8326	0.9832	0.9141	**0.9969**
10	0.6169	0.5507	0.4831	**0.9900**	0.7215	0.5519	0.8962	0.9000	0.2379	0.9356
11	0.3370	0.0000	0.7462	**0.9531**	0.5511	0.0000	0.0000	0.8702	0.0000	0.0000
12	0.0273	0.0047	0.8357	0.0000	0.5051	0.2287	**1.0000**	0.4926	0.8583	0.1012
13	0.3548	0.8134	**0.9582**	0.0000	0.0656	0.0690	0.0000	0.0000	0.0000	0.0000
14	0.5584	0.6512	0.1390	0.0000	0.0013	**0.7632**	0.0000	0.0000	0.0000	0.0000
15	0.4994	0.4279	0.2570	0.8563	0.9477	0.0980	**0.9692**	0.9436	0.8176	0.7881
16	0.0000	0.8321	0.4382	**1.0000**	0.2911	0.0000	**1.0000**	0.9575	0.9723	**1.0000**
OA (%)	0.6799	0.5673	0.5563	0.7881	0.7193	0.5382	0.7361	0.7683	0.7066	**0.8360**
Kappa	0.6572	0.5431	0.5472	0.7602	0.6900	0.4793	0.7106	0.7382	0.6741	**0.8162**
NMI	0.7356	0.6859	0.6631	0.8514	0.8288	0.6589	0.8769	0.8251	0.7841	**0.8587**
ARI	0.5466	0.4693	0.4539	0.7068	0.6278	0.4831	0.7183	0.6532	0.6096	**0.7742**
Purity	0.7132	0.7016	0.6708	0.7893	0.7324	0.5765	0.8357	0.8002	0.7129	**0.8368**

TABLE IV
QUANTITATIVE EXPERIMENTAL CLASSIFICATION RESULTS ON PU

No.	k-means	FCM	PCM	NCSC	DFCN	SDCN	EGAE	AdaGAE	DAEGC	AHSGC
1	0.8234	0.5732	0.5538	0.4238	0.4051	0.6234	0.3029	**0.8521**	0.3759	0.4416
2	0.3581	0.3265	0.4633	0.5933	0.4008	0.3803	0.5012	0.5329	**0.7935**	0.7466
3	0.2167	0.0024	0.6027	0.6902	0.7038	0.1687	**0.9900**	0.7194	0.9609	0.4383
4	0.5943	0.5367	0.1648	0.0270	0.3547	0.4025	0.2744	**0.7237**	0.6505	0.6364
5	0.6620	0.7124	0.0000	0.9735	0.6892	0.6610	**1.0000**	0.0000	**1.0000**	**1.0000**
6	0.4255	0.3529	0.3265	0.9711	0.4940	0.3328	**1.0000**	0.4932	0.3327	0.4979
7	0.0000	**0.9578**	0.0068	0.3382	0.3486	0.3622	0.0000	0.0000	0.5301	0.6474
8	**0.9608**	0.8730	0.5538	0.4027	0.3444	0.4729	0.6258	0.2320	0.1645	0.7588
9	**0.9990**	0.0000	0.9989	0.0000	0.1683	0.3702	0.0000	0.4538	0.0847	0.0000
OA (%)	0.5237	0.4359	0.4376	0.5519	0.4214	0.4207	0.5328	0.5436	0.6010	**0.6365**
Kappa	0.5246	0.4327	0.4261	0.4570	0.3189	0.3185	0.4536	0.4139	0.4866	**0.5429**
NMI	**0.5529**	0.5126	0.4758	0.4392	0.4355	0.4362	0.5372	0.4628	0.5142	0.5251
ARI	0.3188	0.2632	0.2783	0.3869	0.2691	0.2399	0.3780	0.3271	0.5049	**0.5128**
Purity	**0.7012**	0.6785	0.6487	0.6638	0.5966	0.6400	0.7126	0.6689	0.6842	0.6980

noting that DAEGC and AHSGC have achieved breakthrough performance with an OA of 60.10% and 63.65%, respectively. This is mainly because these two methods adopt the structural graph learning mechanism, which can effectively learn the structural information of the graph, that is, have a better understanding of the latent spatial structural information of the graph.

3) Quantitative Analysis of the Trento Dataset: Compared with the SA and PU datasets, the Trento has a relatively simple distribution of land cover types. However, in Table V, the performance of traditional clustering methods, i.e., k-means, FCM, and PCM, is still unsatisfactory, the OAs are only 63.01%, 51.32%, and 36.78%, respectively, owing to their limited feature extraction capability. Notably, AHSGC has achieved commendable results, with an OA of 86.03%, exceeding the second-best performance of 10.41%, which is mainly due to its robust powerful feature extraction and discriminative ability between different types of clusters. Due to the homophily-enhanced structure learning module, AHSGC can estimate the interrelationships between nodes and classes, which enables automatic updating of edge connections between nodes, thus improving clustering accuracy. Additionally, the self-supervised structure and feature loss achieve self-representation feature extraction of HSI.

D. Visual Clustering Results Analysis

To visually display the clustering results, different colormaps are adopted to represent the clustering results of different land features, as shown in Figs. 5–7. From the visualized clustering results, we can conclude that the clustering results of traditional methods, including K-means, FCM, and PCM, contain a lot of salt and pepper noise, indicating insufficient feature extraction capability and poor noise robustness. Benefiting from their nonlinear representation and high-level feature extraction capabilities, deep learning methods achieve smoother colormaps with less noise. However, for specific details such as edge pixels, misclassification may occur. Due

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:56:29 UTC from IEEE Xplore. Restrictions apply.

TABLE V
QUANTITATIVE EXPERIMENTAL CLASSIFICATION RESULTS ON TRENTO

No.	k-means	FCM	PCM	NCSC	DFCN	SDCN	EGAE	AdaGAE	DAEGC	AHSGC
1	0.0000	0.0000	0.3673	**1.0000**	0.6844	0.2218	0.6522	**1.0000**	**1.0000**	0.9995
2	0.0189	0.0000	0.0000	0.0793	0.4191	0.7826	0.6029	0.4826	0.1151	**1.0000**
3	0.0000	0.0000	0.0000	0.0000	0.0063	0.0359	**0.0931**	0.0000	0.0000	0.0000
4	0.7200	0.6826	0.4502	**1.0000**	0.9745	0.6237	0.9540	0.9622	**1.0000**	**1.0000**
5	0.6182	0.2007	0.4292	0.6437	0.7105	0.7452	0.5196	0.6158	0.6156	**0.9394**
6	0.0199	0.0329	0.0006	0.8206	0.3959	0.5930	0.3412	0.8029	**0.8233**	0.0224
OA (%)	0.6301	0.5132	0.3678	0.7489	0.7145	0.6200	0.6487	0.7562	0.7469	**0.8603**
Kappa	0.6026	0.4917	0.3232	0.6790	0.6314	0.5132	0.5380	0.6810	0.6741	**0.8148**
NMI	0.4928	0.4654	0.2898	0.7312	0.6787	0.4860	0.6801	0.7250	0.6767	**0.8844**
ARI	0.3457	0.2802	0.1303	0.7111	0.6795	0.4362	0.6624	0.6455	0.6483	**0.8884**
Purity	0.6499	0.6708	0.5146	0.8652	0.7817	0.7050	0.8273	0.7726	0.8382	**0.8810**

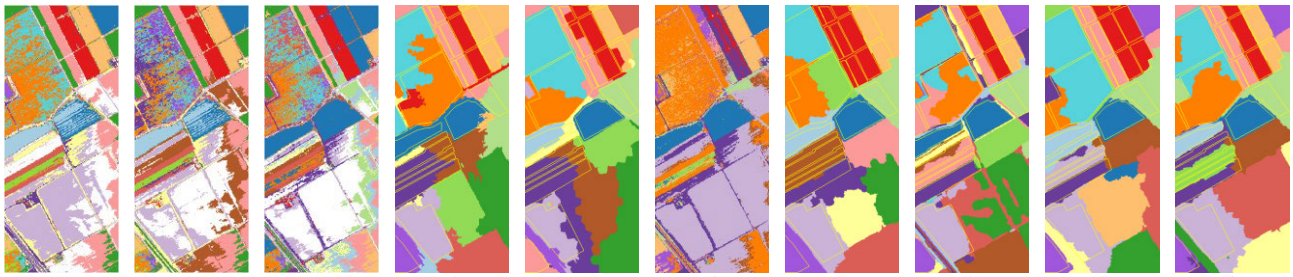


Fig. 5. Clustering maps of SA dataset. (a) K-means, (b) FCM, (c) PCM, (d) NCSC, (e) DFCN, (f) SDCN, (g) EGAE, (h) AdaGAE, (i) DAEGC, and (j) AHSGC.

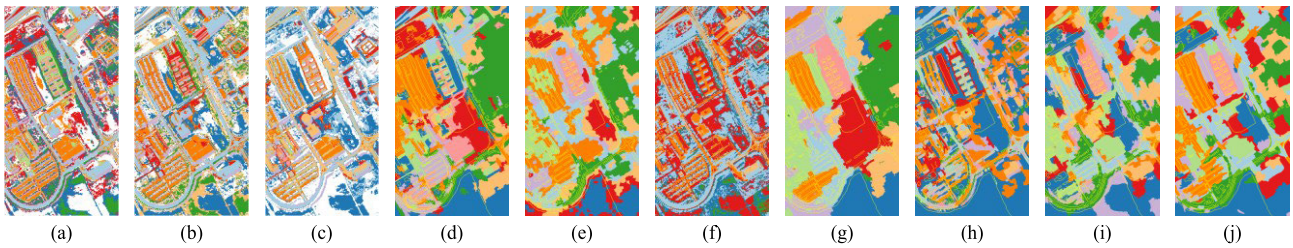


Fig. 6. Clustering maps of PU dataset. (a) K-means, (b) FCM, (c) PCM, (d) NCSC, (e) DFCN, (f) SDCN, (g) EGAE, (h) AdaGAE, (i) DAEGC, and (j) AHSGC.

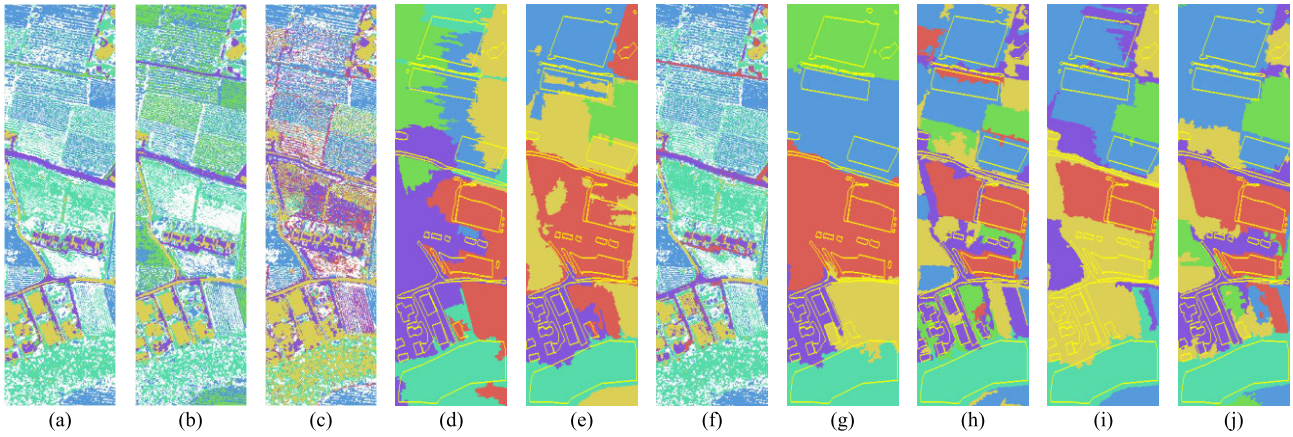


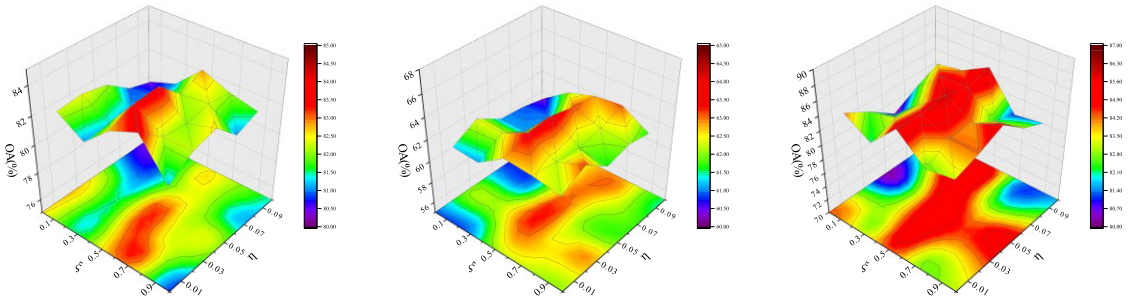
Fig. 7. Clustering maps of Trento dataset. (a) K-means, (b) FCM, (c) PCM, (d) NCSC, (e) DFCN, (f) SDCN, (g) EGAE, (h) AdaGAE, (i) DAEGC, and (j) AHSGC.

to the adaptive filter, AHSGC can effectively capture the low-pass and high-pass information for clustering, and the excess high-frequency and low-frequency noises are filtered out. In addition, the homophily-enhanced structure learning module, by estimating the edge connections between nodes and classes, can update the connections between nodes and

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:56:29 UTC from IEEE Xplore. Restrictions apply.

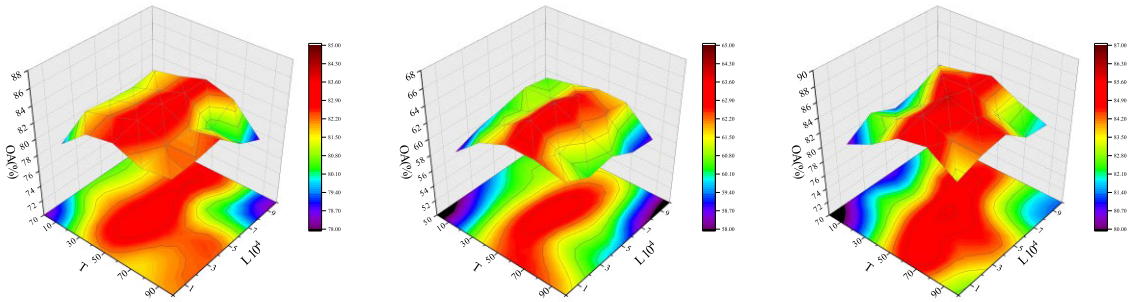
TABLE VI
RUNNING TIME AND MODEL COMPLEXITY OF ALL DEEP CLUSTERING METHODS ON THREE DATASETS

Name	Salinas			PU			Trento		
	Train/s	Test/s	FLOPs	Train/s	Test/s	FLOPs	Train/s	Test/s	FLOPs
AdaGAE	144.8671	43.5324	130.02M	1200.819	101.3368	1.29G	86.2017	13.7605	25.53M
EAGE	199.946	4.8399	270.38M	1763.686	39.8889	2.63G	186.1897	4.7581	57.09M
NCSC	681.0274	15.6768	303.18G	1375.568	33.2784	596.49G	677.5501	19.3583	273.67G
DAEGC	73.4302	5.1500	148.72 M	581.7761	44.8480	1.46 G	65.9399	4.7916	27.75 M
DFCN	672.1873	4.2803	490.69 M	1802.5410	8.5483	1.70 G	282.6037	4.4386	83.27 M
SDCN	437.5881	0.6087	446.1G	274.8834	0.2034	339.56G	145.6749	0.3492	236.22G
AHSGC	179.9025	0.0025	3.57 G	237.5779	0.0007	30.62G	56.2532	0.0629	2.95 G



(a) (b) (c)

Fig. 8. Sensitivities of ξ and η on (a) SA, (b) PU, and (c) Trento.



(a) (b) (c)

Fig. 9. Sensitivities of T and L on (a) SA, (b) PU, and (c) Trento.

connections between classes, with which the perceptual representation of the HSI topology is effectively captured.

E. Complexity Comparison

In this part, we estimate the runtime and model complexity of all investigated deep clustering methods. We record the indexes of training time, testing time, and FLOP to compare the computational cost and model complexity of different methods. According to the results in Table VI, we note that the training, testing time, and model complexity of our method are much lower than other investigated methods, verifying the success of HSI processing. Additionally, due to AHSGC's powerful and fast feature extraction capabilities, requiring only 50 epochs to complete convergence, its training time is relatively short. Furthermore, AHSGC adopts task-oriented self-training and graph structure reconstruction objective loss to accelerate the ability effectively, and the representative features and the computational efficiency of the model are extracted and enhanced, respectively. Therefore, we can conclude that our method achieves remarkable clustering results with a slight model complexity, which indicates that AHSGC contains excellent practical application prospects.

F. Parameter Analysis

In this section, the impacts of different hyperparameters contained in AHSGC on clustering performance are investigated. The number of iterations T, learning rate L, intracluster edge recovery ratio ξ, and intercluster edge removal ratio η are explored through a grid search strategy. Figs. 8 and 9 demonstrate the variations in OA on three datasets with different values of (ξ, η) and (T, L), respectively. From the results, we can observe that four parameters achieve a

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:56:29 UTC from IEEE Xplore. Restrictions apply.

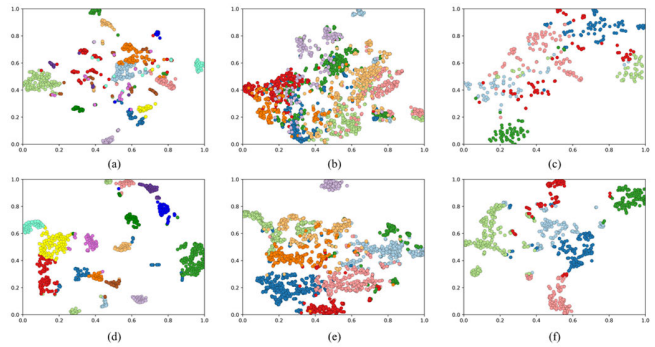


Fig. 10. Visualization of AHSGC using t-SNE on (a) and (d) SA, (b) and (e) PU, and (c) and (f) Trento. (a)–(c) Original distributions of three datasets. (d)–(f) Distributions after clustering. Different color nodes within the maps denote different land covers.

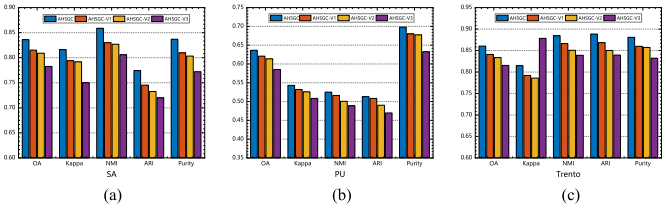


Fig. 11. Ablation experiment results with five metrics on three datasets. (a) On SA, (b) on PU, and (c) on Trento.

significant impact on finally clustering accuracy, and the model is more likely to achieve superior performance when the $\xi \in [0.5, 0.7]$, $\eta \in [0.3, 0.5]$, $T \in [40, 60]$, and $L \in [3e^{-4}, 5e^{-4}]$, respectively.

G. Clustering Visualization

In this section, the t-distributed stochastic neighbor embedding (t-SNE) [43] method is employed to visualize the distribution of graph nodes in three datasets to validate the clustering performance of AHSGC. From Fig. 10(a)–(c) are the original distribution of original graph nodes, while (d)–(f) represent the distribution of graph node features after processing with AHSGC. By comparing and analyzing, the following conclusion can be drawn: 1) the distribution covariance in different classes in (d)–(f) is smaller than that in (a)–(c), indicating that good clustering results have been produced; 2) (d)–(f) demonstrate more considerable interclass distances and smaller intraclass distances, compared with (a)–(c), showing a more regular and compact distribution; 3) Different land covers are distinguished more clearly. Thus, graph nodes processed by AHSGC exhibit tighter intraclass compactness and more considerable interclass distances. Furthermore, the clustering task mentioned in (6) is effectively implemented.

H. Ablation Experiment

In this section, we will evaluate the contribution of the adaptive filter graph encoder, homophily-enhanced structure learning and HSI preprocessing, and initial graph construction module to the overall clustering performance. In addition, a set of ablation studies have been conducted. Specifically, AHSGC without the generation of homogeneous regions block is called AHSGC-V1. AHSGC-V2 is formed by removing the adaptive filter graph encoder block. AHSGC-V3 is obtained by removing the homophily-enhanced structure learning block. The results are shown in Fig. 11 recording five metrics. From the results, we note that the impacts of different modules in AHSGC on clustering accuracy are different. Additionally, each module in the AHSGC contributes to the improvement of clustering accuracy.

V. Conclusion

This article proposes a novel homophily enhanced structure graph learning with an adaptive filter clustering method (AHSGC) for HSI. Specifically, we adopt a homogeneous region generation method to preprocess the HSI, with which the local spatial–spectral structure information is preserved, and the node number in the graph is reduced. After that, to adaptively capture the high and low frequency features on the graph for clustering, an adaptive filter graph encoder is introduced. Then, a graph embedding clustering self-training decoder is developed with KL Divergence, with which the pseudo-label is generated for network training. Meanwhile, we design a homophily enhanced structure graph learning module to update the graph according to the task, in which the orient correlation estimation is adopted to estimate the node connection, and graph edge sparsification is designed to adjust the edges in the graph dynamically. Finally, the joint network optimization is proposed to optimize the network, and the K-means is adopted to express the latent features.

In future work, some more spatial–spectral self-supervised methods will be explored for graph learning. In addition, some more research, e.g., reinforcement learning, will be applied for HSI preprocessing to enhance the feature extraction abilities of clustering methods.

References

[1] J. Peng, W. Sun, and Q. Du, "Self-paced joint sparse representation for the classification of hyperspectral images," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 2, pp. 1183–1194, Feb. 2019.

[2] J. Peng, L. Li, and Y. Y. Tang, "Maximum likelihood estimation-based joint sparse representation for the classification of hyperspectral remote sensing images," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 6, pp. 1790–1802, Jun. 2019.

[3] Z. Lv, H. Huang, W. Sun, T. Lei, J. Atli Benediktsson, and J. Li, "Novel enhanced UNet for change detection using multimodal remote sensing image," *IEEE Geosci. Remote Sens. Lett.*, vol. 20, pp. 1–5, 2023.

[4] Z. Lv, J. Liu, W. Sun, T. Lei, J. A. Benediktsson, and X. Jia, "Hierarchical attention feature fusion-based network for land cover change detection with homogeneous and heterogeneous remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, pp. 1–15, 2023, Art. no. 4411115.

[5] J. Feng, G. Bai, D. Li, X. Zhang, R. Shang, and L. Jiao, "MR-selection: A meta-reinforcement learning approach for zero-shot hyperspectral band selection," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, pp. 1–20, 2023, Art. no. 5500320.

[6] J. Feng et al., "Convolutional neural network based on bandwise-independent convolution and hard thresholding for hyperspectral band selection," *IEEE Trans. Cybern.*, vol. 51, no. 9, pp. 4414–4428, Sep. 2021.

[7] D. Yao et al., "Deep hybrid: Multi-graph neural network collaboration for hyperspectral image classification," *Defence Technol.*, vol. 23, pp. 164–176, May 2023.

[8] C. Lou et al., "Land use/land cover (LULC) classification using hyperspectral images: A review," *Geo-Spatial Inf. Sci.*, vol. 2024, pp. 1–42, Apr. 2024.

[9] Y. Cai, Z. Zhang, X. Liu, Y. Ding, F. Li, and J. Tan, "Learning unified anchor graph for joint clustering of hyperspectral and LiDAR data," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, May 6, 2024, doi: 10.1109/TNNLS.2024.3392484.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:56:29 UTC from IEEE Xplore. Restrictions apply.

[10] Y. Ding, X. Zhao, Z. Zhang, W. Cai, and N. Yang, "Graph sample and aggregate-attention network for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.

[11] Y. Ding et al., "Unsupervised self-correlated learning smoothy enhanced locality preserving graph convolution embedding clustering for hyperspectral images," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5536716.

[12] D. Al-Alimi, M. A. A. Al-Qaness, Z. Cai, and E. A. Alawamy, "IDA: Improving distribution analysis for reducing data complexity and dimensionality in hyperspectral images," *Pattern Recognit.*, vol. 134, Feb. 2023, Art. no. 109096.

[13] T. Kanungo, D. M. Mount, N. S. Netanyahu, C. D. Piatko, R. Silverman, and A. Y. Wu, "An efficient k-means clustering algorithm: Analysis and implementation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 24, no. 7, pp. 881–892, Jul. 2002.

[14] X. Yang, M. Zhu, B. Sun, Z. Wang, and F. Nie, "Fuzzy C-multiple-means clustering for hyperspectral image," *IEEE Geosci. Remote Sens. Lett.*, vol. 20, pp. 1–5, 2023.

[15] A. Y. Ng, M. I. Jordan, and Y. Weiss, "On spectral clustering: Analysis and an algorithm," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 14, Jan. 2001, pp. 849–856.

[16] R. Krishnapuram and J. M. Keller, "The possibilistic C-means algorithm: Insights and recommendations," *IEEE Trans. Fuzzy Syst.*, vol. 4, no. 3, pp. 385–393, Aug. 1996.

[17] A. Rodriguez and A. Laio, "Clustering by fast search and find of density peaks," *Science*, vol. 344, no. 6191, pp. 1492–1496, Jun. 2014.

[18] Y. Cai et al., "Superpixel contracted neighborhood contrastive subspace clustering network for hyperspectral images," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5530113.

[19] X. Peng, H. Tang, L. Zhang, Z. Yi, and S. Xiao, "A unified framework for representation-based subspace clustering of out-of-sample and large-scale data," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 27, no. 12, pp. 2499–2512, Dec. 2016.

[20] N. Gillis, D. Kuang, and H. Park, "Hierarchical clustering of hyperspectral images using rank-two nonnegative matrix factorization," *IEEE Trans. Geosci. Remote Sens.*, vol. 53, no. 4, pp. 2066–2078, Apr. 2015.

[21] J. W. P. Sun, H. Li, W. Li, X. Meng, C. Ge, and Q. Du, "Low-rank and sparse representation for hyperspectral image processing: A review," *IEEE Geosci. Remote Sens. Mag.*, vol. 10, no. 1, pp. 10–43, Jun. 2021.

[22] Y. Su, L. Gao, M. Jiang, A. Plaza, X. Sun, and B. Zhang, "NSCKL: Normalized spectral clustering with kernel-based learning for semisupervised hyperspectral image classification," *IEEE Trans. Cybern.*, vol. 53, no. 10, pp. 6649–6662, Oct. 2022.

[23] H. Zhang, L. Bruzzone, W. Shi, M. Hao, and Y. Wang, "Enhanced spatially constrained remotely sensed imagery classification using a fuzzy local double neighborhood information C-means clustering algorithm," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 11, no. 8, pp. 2896–2910, Aug. 2018.

[24] H. Zhai, H. Zhang, L. Zhang, P. Li, and A. Plaza, "A new sparse subspace clustering algorithm for hyperspectral remote sensing imagery," *IEEE Geosci. Remote Sens. Lett.*, vol. 14, no. 1, pp. 43–47, Jan. 2017.

[25] R. Wang, F. Nie, Z. Wang, F. He, and X. Li, "Scalable graph-based clustering with nonnegative relaxation for large hyperspectral image," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 10, pp. 7352–7364, Oct. 2019.

[26] Y. Ding et al., "Multi-feature fusion: Graph neural network and CNN combining for hyperspectral image classification," *Neurocomputing*, vol. 501, pp. 246–257, Aug. 2022.

[27] Y. Zhang, Y. Wang, X. Chen, X. Jiang, and Y. Zhou, "Spectral–spatial feature extraction with dual graph autoencoder for hyperspectral image clustering," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 32, no. 12, pp. 8500–8511, Dec. 2022.

[28] Y. Ding et al., "Self-supervised locality preserving low-pass graph convolutional embedding for large-scale hyperspectral image clustering," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–16, 2022, Art. no. 5536016.

[29] F. Luo, Y. Liu, Y. Duan, T. Guo, L. Zhang, and B. Du, "SDST: Self-supervised double-structure transformer for hyperspectral images clustering," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, pp. 1–14, 2024, Art. no. 5511614.

[30] C. Wang, S. Pan, R. Hu, G. Long, J. Jiang, and C. Zhang, "Attributed graph clustering: A deep attentional embedding approach," 2019, *arXiv:1906.06532*.

[31] S. Peng, J. Yin, Z. Yang, B. Chen, and Z. Lin, "Multiview clustering via hypergraph induced semi-supervised symmetric nonnegative matrix factorization," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 33, no. 10, pp. 5510–5524, Oct. 2023.

[32] Z. Chen, G. Wu, H. Gao, Y. Ding, D. Hong, and B. Zhang, "Local aggregation and global attention network for hyperspectral image classification with spectral-induced aligned superpixel segmentation," *Expert Syst. Appl.*, vol. 232, Dec. 2023, Art. no. 120828.

[33] M. Gu et al., "Homophily-enhanced structure learning for graph clustering," in *Proc. 32nd ACM Int. Conf. Inf. Knowl. Manage.*, Oct. 2023, pp. 577–586.

[34] Z. Wen et al., "Homophily-related: Adaptive hybrid graph filter for multi-view graph clustering," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 15841–15849.

[35] R. Achanta, A. Shaji, K. Smith, A. Lucchi, P. Fua, and S. Süsstrunk, "SLIC superpixels compared to state-of-the-art superpixel methods," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 34, no. 11, pp. 2274–2282, Nov. 2012.

[36] W. Yang and B. Mirzasoleiman, "Contrastive learning under heterophily," 2023, *arXiv:2303.06344*.

[37] E. Pan and K. Zhao, "Beyond homophily: Reconstructing structure for graph-agnostic clustering," in *Proc. Int. Conf. Mach. Learn.*, Jan. 2023, pp. 26868–26877.

[38] Y. Liu et al., "Simple contrastive graph clustering," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 10, pp. 13789–13800, Oct. 2024.

[39] W. Tu et al., "Deep fusion clustering network," in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 35, no. 11, pp. 9978–9987.

[40] D. Bo, X. Wang, C. Shi, M. Zhu, E. Lu, and P. Cui, "Structural deep clustering network," in *Proc. Web Conf.*, Apr. 2020, pp. 1400–1410.

[41] H. Zhang, P. Li, R. Zhang, and X. Li, "Embedding graph auto-encoder for graph clustering," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 11, pp. 9352–9362, Nov. 2022.

[42] X. Li, H. Zhang, and R. Zhang, "Adaptive graph auto-encoder for general data clustering," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 12, pp. 9725–9732, Dec. 2022.

[43] L. Van der Maaten and G. Hinton, "Visualizing data using t-SNE," *J. Mach. Learn. Res.*, vol. 9, no. 11, pp. 1–27, 2008.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 19:56:29 UTC from IEEE Xplore. Restrictions apply.